Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S06: Hands-On Lab Walkthrough

Kafka Connect Ingestion, SMT Regex Routing, Sink Egress & Dead Letter Queues

Lab Objectives

In this hands-on terminal lab, you will:
1. Probe the **Kafka Connect REST API** and verify loaded connector plugins.
2. Ensure integration and error quarantine topics are provisioned on the Kafka cluster.
3. Deploy a **FileStreamSourceConnector** that continuously streams order events from `/data/orders.jsonl`.
4. Apply an in-flight **Single Message Transform (SMT: `RegexRouter`)** to dynamically retarget events from `orders.staging.v1` to `orders.copy.v1`.
5. Deploy a **FileStreamSinkConnector** configured with **Dead Letter Queue (DLQ)** error tolerance (`orders.connect.dlq`).
6. Verify end-to-end data egress into `/data/sink-orders.txt` and monitor task offset commits.
7. Run the certified test suite `verify_s06.sh` to validate all 5 verification gates.

Step 1: Pre-Flight Environment & REST API Inspection

Navigate to the lab workspace:

```
cd ~/Desktop/Work/Training/Spark/00_Lab
tools/check_stack.sh
```

Query the Kafka Connect REST API on port `8083` to list all available plugins:

```
curl -s http://localhost:8083/connector-plugins | jq .
```

Expected output:

```
[
  {
    "class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
    "type": "sink",
    "version": "8.4.0-ccs"
  },
  {
    "class": "org.apache.kafka.connect.file.FileStreamSourceConnector",
    "type": "source",
    "version": "8.4.0-ccs"
  },
  ...
]
```

Step 2: Provision Required Integration Topics

Ensure the destination and DLQ topics exist on the 3-broker KRaft cluster:

```
docker exec kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --create --if-not-exists --topic orders.copy.v1 --partitions 3 --replication-factor 3

docker exec kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --create --if-not-exists --topic orders.connect.dlq --partitions 1 --replication-factor 3
```

Step 3: Inspect & Deploy Source Connector with SMT

Inspect the connector configuration at `connectors/file_source.json`:

```
cat connectors/file_source.json | jq .
```

> **Key SMT Observation:** Notice how the `RegexRouter` transform intercepts the configured topic `orders.staging.v1` and rewrites it to `orders.copy.v1`:
>
> ```
> json
> "transforms": "route",
> "transforms.route.type": "org.apache.kafka.connect.transforms.RegexRouter",
> "transforms.route.regex": "orders\\.staging\\.v1",
> "transforms.route.replacement": "orders.copy.v1"
> ```

Register the source connector via HTTP `POST`:

```
curl -s -X POST -H "Content-Type: application/json" \
  --data @connectors/file_source.json http://localhost:8083/connectors | jq .
```

Query the status of the newly created connector:

```
curl -s http://localhost:8083/connectors/orders-file-source/status | jq .
```

Expected output: Both `connector.state` and `tasks[0].state` must report `RUNNING`.

Verify that records have been routed to `orders.copy.v1`:

```
docker exec kafka1 /opt/kafka/bin/kafka-get-offsets.sh \
  --bootstrap-server kafka1:9092 --topic orders.copy.v1 --time -1
```

Expected output: Offsets across partitions 0, 1, and 2 should reflect ingested order lines.

Step 4: Inspect & Deploy Sink Connector with Dead Letter Queue

Inspect `connectors/file_sink.json`:

```
cat connectors/file_sink.json | jq .
```

> **DLQ Configuration Highlights:**
> - `errors.tolerance: all`: Prevents task crashes on corrupt records.
> - `errors.deadletterqueue.topic.name: orders.connect.dlq`: Isolates poison pills.
> - `errors.deadletterqueue.context.headers.enable: true`: Injects error metadata into message headers.

Register the sink connector:

```
curl -s -X POST -H "Content-Type: application/json" \
  --data @connectors/file_sink.json http://localhost:8083/connectors | jq .
```

Verify task status:

```
curl -s http://localhost:8083/connectors/orders-file-sink/status | jq .
```

Expected output:

```
{
  "name": "orders-file-sink",
  "connector": {"state": "RUNNING", "worker_id": "connect:8083"},
  "tasks": [{"id": 0, "state": "RUNNING", "worker_id": "connect:8083"}],
  "type": "sink"
}
```

Step 5: Verify File Egress & Offset Tracking

Check that the sink file `/data/sink-orders.txt` has been written:

```
docker exec connect ls -lh /data/sink-orders.txt
docker exec connect head -n 5 /data/sink-orders.txt
```

Count the total records written to the destination file:

```
docker exec connect wc -l /data/sink-orders.txt
```

Expected output: Over 4,000 order records successfully ingested from source file, routed through Kafka, and written out to the sink file!

Check the consumer group offsets committed by the sink task:

```
docker exec kafka1 /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka1:9092 --describe --group connect-orders-file-sink
```

Step 6: Automated Verification Gate Execution

Run the certified test runner to validate all 5 verification gates:

```
bash ../S06_Kafka_Connect_Data_Integration/Trainer/verify_s06.sh
```

Expected Terminal Result:

```
Running S06 Automated Kafka Connect Verification Gate...
[1/5] Probing Kafka Connect REST endpoint (http://localhost:8083)... PASS (HTTP 200 OK & Fil
eStream plugins loaded)
[2/5] Verifying required integration topics exist... PASS (orders.copy.v1 & orders.connect.d
lq ready)
[3/5] Registering Source Connector with SMT RegexRouter... PASS (Source registered, SMT acti
ve, task state: RUNNING)
[4/5] Registering Sink Connector with Dead Letter Queue (DLQ)... PASS (Sink registered, DLQ
configured, task state: RUNNING)
[5/5] Verifying end-to-end data flow (File → SMT → Kafka → DLQ/Sink)... PASS (4108 record
s ingested, routed & sinked successfully)

★ PASS: S06 Lab Requirements 100% Satisfied! ★
```

🌟 Gold Stretch Challenge: Chained SMT Pipeline & Dead Letter Topic

> **Distinguished Engineer Challenge:** If you complete the core lab before time expires, construct an enterprise chained Single Message Transform (SMT) pipeline:
> 1. Configure a sink connector with chained transforms:
> - Apply `TimestampConverter` to convert ISO-8601 string dates into epoch milliseconds.
> - Apply `MaskField` to hash sensitive customer PII before writing downstream.
> 2. Inject a malformed poison-pill message into the source topic and verify that Kafka Connect routes the failed record into `retail.orders.dlq` without stopping the connector worker task.